

Container queries

In the last lesson, media queries changed the page when the **viewport** reached a certain width. That works when the whole page needs to respond to the **browser size**, but sometimes a component needs to respond to the space it gets inside a container.

Imagine the same lesson card appears in two places:

- A narrow sidebar.
- A wider main content area.

The browser viewport might be wide, but the sidebar card is still narrow. A media query can only ask, "How wide is the viewport?" A container query can ask, "How wide is the space around this component?" That is the point of container queries.

Start with the component problem

To see the problem, build a page where the same card component appears in two different places. One card lives in a narrow sidebar. Another card lives in a wider main area.

Use this HTML for the experiment:

html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Container query practice</title>
    <link rel="stylesheet" href="style.css">
  </head>
  <body>
    <main class="page">
      <section class="intro">
        <p class="eyebrow">CSS practice</p>
        <h1>Make a card respond to its container</h1>
        <p class="summary">The same component can have different amounts of space depend
      </section>

      <section class="layout">
        <aside class="sidebar">
          <h2>Sidebar</h2>

          <div class="card-shell">
            <article class="lesson-card">
              
                <p class="card-label">Featured</p>
                <h3>Container queries</h3>
                <p>Adapt a component based on the space around it, not only the viewport
```



```

        <a class="card-link" href="/container-queries">Open lesson</a>
      </div>
    </article>
  </div>
</aside>

<section class="workspace">
  <h2>Main area</h2>

  <div class="card-shell">
    <article class="lesson-card">
      
        <p class="card-label">Featured</p>
        <h3>Container queries</h3>
        <p>Adapt a component based on the space around it, not only the viewport
        <a class="card-link" href="/container-queries">Open lesson</a>
      </div>
    </article>
  </div>
</section>
</section>
</main>
</body>
</html>
```

Now add the base CSS:

CSS

```
* {
  box-sizing: border-box;
}
```

```
body {
```

```
  background-color: #f9fafb;
}

.page {
  max-width: 1040px;
  margin: 40px auto;
  padding: 0 16px;
}

.intro,
.sidebar,
.workspace,
.lesson-card {
  background-color: white;
  border: 1px solid #e5e7eb;
  border-radius: 8px;
}

.intro,
.sidebar,
.workspace {
  padding: 24px;
}

.intro {
  margin-bottom: 24px;
}

.eyebrow,
.card-label {
  color: #2563eb;
  font-size: 14px;
  font-weight: 700;
}

h1,
h2,
```

```
h3,
p {
  margin-top: 0;
}

.summary,
.lesson-card p {
  color: #4b5563;
  line-height: 1.6;
}

.layout {
  display: flex;
  gap: 24px;
  align-items: flex-start;
}

.sidebar {
  width: 280px;
}

.workspace {
  flex: 1;
}

.lesson-card {
  overflow: hidden;
}

.card-image {
  display: block;
  width: 100%;
  height: auto;
}

.card-body {
  padding: 18px;
}

.card-link {
  color: #2563eb;
  font-weight: 700;
  text-decoration-thickness: 2px;
  text-decoration-offset: 3px;
}
```

Open the page. The two cards use the same HTML and the same CSS, but they sit in different spaces: the sidebar card is narrow, and the main-area card has more room.

Right now both cards use the same stacked layout: image on top, text below. That is safe for narrow spaces, but the wide card could use its space better.

A few base rules create that experiment:

- **.layout** uses Flexbox only to place the sidebar and workspace beside each other.
- **.sidebar** uses **width: 280px** so one card has a narrow container.
- **.workspace** uses **flex: 1** so the other card gets the remaining space.
- **.lesson-card** uses **overflow: hidden** so the image respects the rounded card corners.

- `.card-image` uses `display: block`, `width: 100%`, and `height: auto` so the image fits the stacked card before any container query runs.

Those rules set up two different component widths. The container query will decide how the card should behave inside each one.

Why a media query is not enough

You might try a media query like this:

CSS

```
@media (min-width: 800px) {  
  .lesson-card {  
    display: flex;  
  }  
  
  .card-image,  
  .card-body {  
    flex: 1;  
  }  
}
```

This asks a viewport question: "Is the browser at least `800px` wide?"

On a wide browser, the answer is yes. Both cards match the rule, including the sidebar card, and that is the problem. The sidebar card is still narrow even when the viewport is wide.

The component does not need to know the viewport width. It needs to know the width of the space it was placed in.

 **Viewport size and component size are different** - A wide browser does not mean every component has wide space. Sidebars, cards, panels, and columns can all be narrow inside a wide viewport.

Container queries have two parts

A container query needs two pieces:

1. A container to measure.
2. A query that runs when that container reaches a size.

The container is usually a wrapper around the component. In this HTML, `.card-shell` wraps each `.lesson-card`:

html

```
<div class="card-shell">
  <article class="lesson-card">
    ...
  </article>
</div>
```

That wrapper is useful because it gives CSS a clear box to measure. The card can then change its internal layout based on the wrapper's width.

The wrapper also avoids a common confusion: a container query styles elements based on an ancestor container. The card should not try to query itself and style itself at the same time. A small wrapper makes the relationship clear: `.card-shell` is measured, and `.lesson-card` changes.

Mark the container

Before CSS can query an element's size, you need to mark that element as a query container.

Mark `.card-shell` as the container:

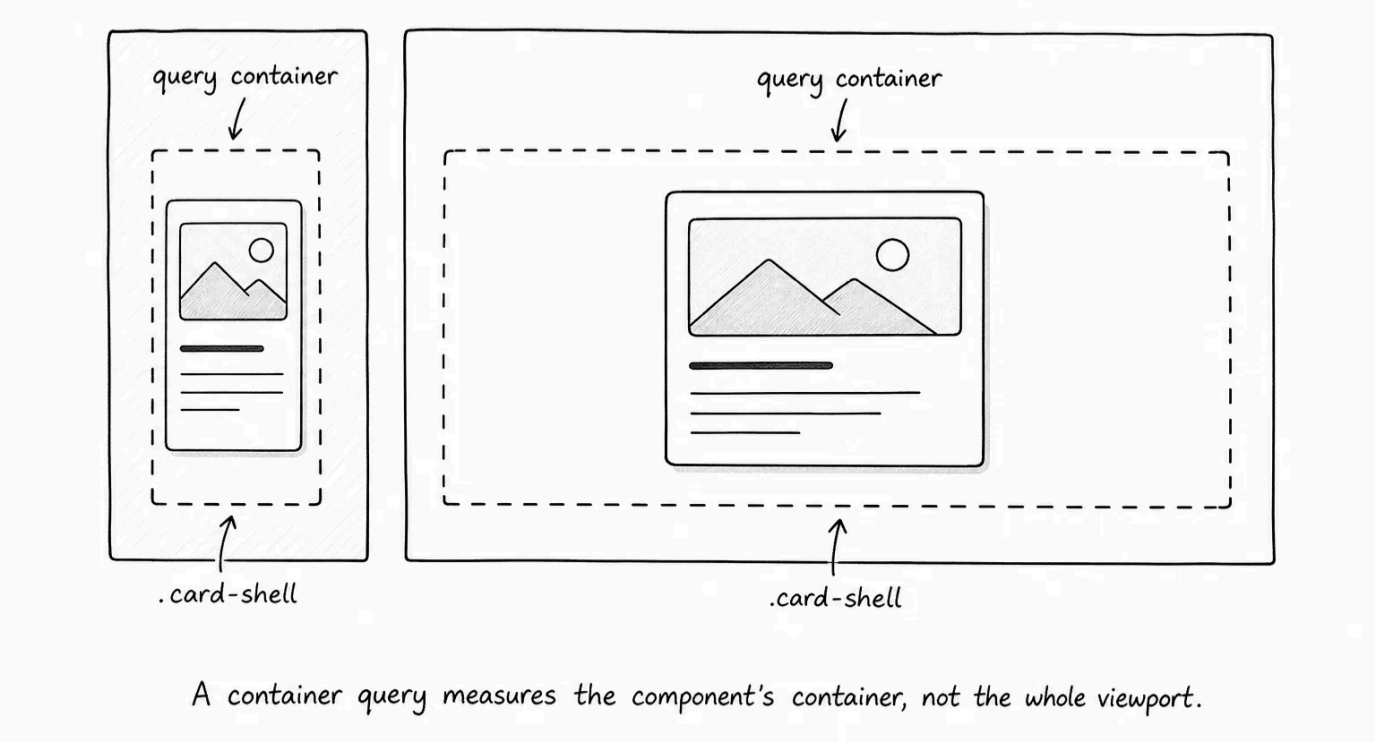
CSS

```
.card-shell {
  container-type: inline-size;
}
```

`container-type: inline-size` says this element can be queried by its inline size.

In normal English-language pages, inline size means width. So for this lesson, read it as: "allow CSS to query how wide `.card-shell` is."

This rule does not style the card visually by itself. It gives the browser permission to use `.card-shell` as a measurable container.



Query the container

Now that `.card-shell` is a query container, CSS can ask whether that container is wide enough for a different card layout.

Add the container query:

CSS

```
@container (min-width: 520px) {  
  .lesson-card {  
    display: flex;  
  }  
  
  .card-image {  
    width: 42%;  
    object-fit: cover;  
  }  
  
  .card-body {  
    flex: 1;  
  }  
}
```

`@container (min-width: 520px)` means "use these rules when the nearest query container is at least `520px` wide."

The sidebar `.card-shell` is narrower than `520px`, so its card stays stacked.

The workspace `.card-shell` is wider than `520px`, so its card becomes a side-by-side layout.

Inside the query, `.lesson-card` becomes a flex container. The image and body become flex items.

`width: 42%` gives the image a clear share of the row. It is not exactly half because text often needs a little more room than the image.

`object-fit: cover` keeps the image from looking stretched when it shares a row with the text. The image may crop slightly, but it keeps its proportions.

`flex: 1` lets `.card-body` take the remaining space.

This is the core container-query habit: let the component stay simple in narrow containers, then enhance it when its own container has enough room.

Container queries are component breakpoints

Media queries create viewport breakpoints, while container queries create component breakpoints.

That difference is practical because the two tools ask different questions:

Question	Use
Is the browser wide enough for the whole page layout to change?	Media que
Is this card's container wide enough for the card layout to change?	Container
Should the header, page shell, or main navigation change?	Media que
Should a reusable card, panel, or widget change based on where it appears?	Container

A container query does not replace every media query; it solves a different problem.

Use container queries for reusable components - If the same component can appear in a narrow sidebar, a wide content area, or a dashboard panel, a container query often fits better than a viewport media query.

Name a container when needed

The query above uses the nearest query container. That is enough for many component examples.

Sometimes a component sits inside several containers, and you want to query a specific one. You can give a container a name:

CSS

```
.card-shell {
  container-type: inline-size;
  container-name: card;
}
```

Then query that named container:

CSS

```
@container card (min-width: 520px) {
  .lesson-card {
    display: flex;
  }
}
```

container-name gives the query container a label. **@container card (...)** says the query should use a container named **card**.

You do not need a name for every container. Use names when they make the query clearer or prevent CSS from accidentally using the wrong ancestor.

Inspect the container

DevTools can help you confirm whether your query is working.

Inspect the wide card and look at the matched CSS rules. When the `.card-shell` is wide enough, the rules inside `@container (min-width: 520px)` should appear.

Then inspect the sidebar card. The same `@container` rule should not apply if the sidebar `.card-shell` is too narrow.

If the query does not work, check these things:

- Did you put `container-type` on an ancestor of the element you are styling?
- Is the container actually wide enough to match the query?
- Are you accidentally querying the viewport with `@media` instead of querying the container with `@container` ?
- If you used `container-name` , does the name match?

Container queries are easiest to debug when you inspect the component and compare the container width with the query condition.

Put the container query rules together

Here are the container-query rules you added:

CSS

```
.card-shell {
  container-type: inline-size;
}

@container (min-width: 520px) {
  .lesson-card {
    display: flex;
  }

  .card-image {
    width: 42%;
    object-fit: cover;
  }

  .card-body {
    flex: 1;
  }
}
```

The pattern is: wrap the component, mark the wrapper as a query container, then write a container query for the layout the component should use when that wrapper has enough space.

Try it

Test the card by changing the container, not the viewport:

- Change `.sidebar` from `width: 280px;` to `width: 560px;` . Notice whether the sidebar card now matches the container query.
- Change the query from `@container (min-width: 520px)` to `@container (min-width: 700px)` . Notice when the wide card stops using the side-by-side layout.
- Remove `container-type: inline-size;` from `.card-shell` . Confirm that the container query no longer works.
- Put `container-type: inline-size;` on `.lesson-card` instead of `.card-shell` . Notice whether the query behaves the way you expected.
- Add a second card inside `.workspace` and check whether the same component rules still apply.
- Try the named-container version with `container-name: card` , then update the query to `@container card (min-width: 520px)` .

The goal is to make component layout depend on component space. If the same component can live in different-sized places, a container query can often make it adapt more naturally than a media query.

< 0 / 10 >

✔ Knew 0 Items

✳️ Learnt 0 Items

⏮ Skipped 0 Items

🔄 ResetProgress

Test Your Understanding

What problem do container queries solve?

[Click to Reveal the Answer](#)

✔ Already Know that

✳️ Didn't Know that

⏮ Skip Question

✓

LESSON COMPLETE

Nicely done.

Up next: Personal Portfolio

Next Lesson →